

ctx Codebase Analysis

A Comprehensive Technical Review of the Context Generation and Code Intelligence CLI Tool

Generated by Automated Code Analysis

Using ctx code intelligence features

December 13, 2025

Abstract

This document provides a comprehensive analysis of the **ctx** codebase, a Rust-based command-line tool for generating AI-ready context from codebases and providing code intelligence features including indexing, search, call graphs, and impact analysis. The analysis covers architecture, module responsibilities, code quality metrics, dependency relationships, and recommendations for improvement. The codebase consists of **548 symbols** across **20 source files** with **2664 edges** representing function calls and relationships.

Contents

1 Executive Summary

ctx is a well-architected Rust command-line tool providing two main capabilities:

1. **Context Generation** — Formats source code for consumption by Large Language Models (LLMs), with intelligent filtering, multiple output formats, and streaming support.
2. **Code Intelligence** — Provides comprehensive code analysis including indexing, full-text search, semantic search with embeddings, call graph generation, and impact analysis.

1.1 Key Metrics

Metric	Value
Total Source Files	20
Total Symbols	548
Functions	446
Structs	35
Enums	11
Traits	3
Total Edges (Relationships)	2,664

Table 1: Codebase statistics overview

1.2 Technology Stack

The tool leverages several key technologies:

- **Rust** — Primary implementation language, providing memory safety and performance
- **Tree-sitter** — Incremental parsing library for accurate AST extraction across multiple languages
- **SQLite** — Persistent storage for symbols, edges, and compressed source code
- **DuckDB** — In-memory analytical database for recursive graph queries
- **FTS5** — Full-text search extension for SQLite
- **fastembed** — Local embedding generation using all-MiniLM-L6-v2
- **clap** — Command-line argument parsing

1.3 Supported Languages

Language	Extensions	Parser	Status
Rust	.rs	tree-sitter-rust	Full
TypeScript	.ts, .tsx	tree-sitter-typescript	Full
JavaScript	.js, .jsx, .mjs	tree-sitter-javascript	Full
Python	.py, .pyi	tree-sitter-python	Full
Solidity	.sol	tree-sitter-solidity	Full
YAML	.yaml, .yml	N/A	File tracking only

Table 2: Supported programming languages

1.4 Quality Assessment Summary

The codebase demonstrates solid Rust practices with clear module separation. However, analysis revealed areas requiring attention:

- **17 high-complexity functions** exceeding recommended fan-out thresholds
- **13 duplicate code pairs** indicating potential for abstraction
- **Clean module separation** with well-defined responsibilities
- **Comprehensive test coverage** for core functionality

2 Architecture

2.1 High-Level Overview

The ctx architecture follows a modular design with clear separation between context generation and code intelligence features. The system is organized into distinct layers, each with well-defined responsibilities.

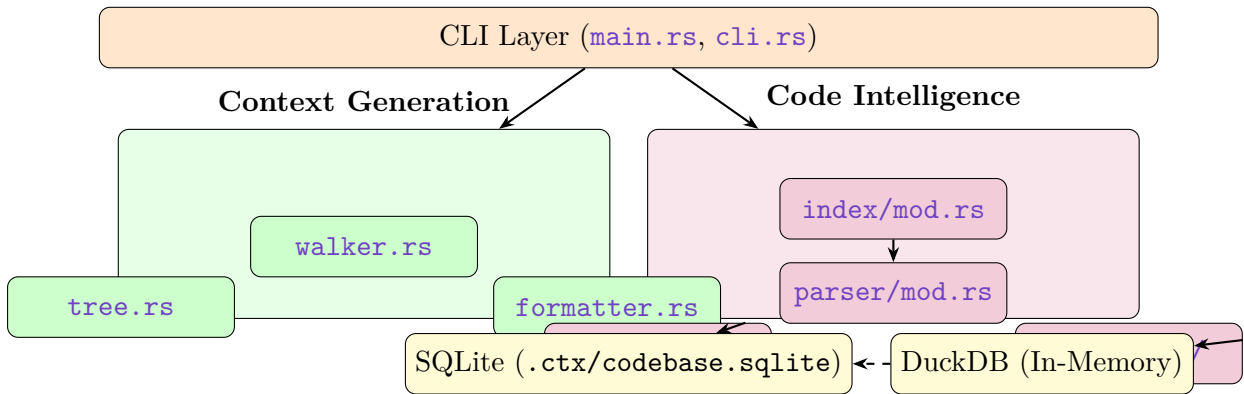


Figure 1: High-level architecture diagram

2.2 Storage Strategy

The system employs a dual-database architecture:

2.2.1 SQLite — Persistent Storage

All persistent data resides in a single SQLite file at `.ctx/codebase.sqlite`:

```

1  -- Files table
2  CREATE TABLE files (
3      path TEXT PRIMARY KEY,
4      content_hash TEXT NOT NULL,
5      size_bytes INTEGER,
6      language TEXT,
7      last_indexed INTEGER,
8      source_compressed BLOB -- gzip compressed
9  );
10
11 -- Symbols table
12 CREATE TABLE symbols (
13     id TEXT PRIMARY KEY,          -- "file_path::name::line"
14     name TEXT NOT NULL,

```

```

15     kind TEXT NOT NULL,           -- function, struct, enum, etc.
16     file_path TEXT NOT NULL,
17     line_start INTEGER,
18     line_end INTEGER,
19     visibility TEXT DEFAULT 'private',
20     signature TEXT,
21     brief TEXT,                   -- First line of docstring
22     parent_id TEXT,
23     FOREIGN KEY (file_path) REFERENCES files(path)
24 );
25
26 -- Edges table (call graph)
27 CREATE TABLE edges (
28     source_id TEXT NOT NULL,       -- Caller symbol ID
29     target_name TEXT NOT NULL,     -- Called function name
30     target_id TEXT,               -- Resolved symbol ID (if found)
31     kind TEXT NOT NULL,          -- 'calls', 'imports', etc.
32     line INTEGER,
33     context TEXT,                -- Code snippet
34     FOREIGN KEY (source_id) REFERENCES symbols(id)
35 );

```

Listing 1: Core database schema

Benefits of SQLite:

- Single file for easy management, backup, and deletion
- ACID transactions for safe concurrent access
- Built-in FTS5 for full-text search
- Widely supported with excellent tooling
- Optimized for insert-heavy indexing workloads

2.2.2 DuckDB — Analytical Queries

For complex analytical queries requiring recursive CTEs and graph traversal, an in-memory DuckDB instance attaches to the SQLite database:

```

1 // Open in-memory DuckDB
2 let conn = Connection::open_in_memory()?;
3
4 // Attach SQLite database read-only
5 conn.execute(
6     "ATTACH 'codebase.sqlite' AS code (TYPE sqlite, READ_ONLY)",
7     [],
8 );
9
10 // Run recursive graph queries
11 conn.execute("
12     WITH RECURSIVE call_graph AS (
13         SELECT source_id, target_id, 1 as depth
14         FROM code.edges WHERE source_id = ?
15         UNION ALL
16         SELECT e.source_id, e.target_id, cg.depth + 1
17         FROM code.edges e
18         JOIN call_graph cg ON e.source_id = cg.target_id
19         WHERE cg.depth < ?

```

```

20     )
21     SELECT * FROM call_graph
22 ", params)?;

```

Listing 2: DuckDB integration pattern

Benefits of DuckDB:

- Columnar storage efficient for analytical queries
- Native support for recursive CTEs
- Direct SQLite attachment without ETL
- In-memory operation with no additional files
- OLAP-optimized for aggregations and joins

2.3 Parsing Strategy

The system uses Tree-sitter for parsing, providing:

- **Speed** — Incremental parsing handles large files efficiently
- **Accuracy** — Real parser, not regex-based heuristics
- **Consistency** — Same API across all supported languages
- **Error Tolerance** — Parses incomplete or invalid code gracefully
- **Battle-tested** — Used by GitHub, Neovim, Zed, and other major tools

2.4 Symbol Identification

Symbols are identified by a composite key ensuring uniqueness:

```
file_path::name::line
```

For example:

```
src/auth/handler.ts::handleAuth::45
src/parser/rust.rs::extract_symbols::200
```

This scheme handles:

- Same function name in different files
- Overloaded functions
- Nested functions and closures

2.5 Incremental Indexing

File changes are tracked using content hashes rather than timestamps:

```

1 fn needs_update(&self, path: &str, content: &str) -> bool {
2     let new_hash = sha256(content);
3     let old_hash = self.db.get_hash(path);
4     new_hash != old_hash
5 }

```

Listing 3: Incremental update detection

This approach ensures:

- Only changed files are re-parsed
- Unchanged files are skipped instantly
- More reliable than timestamp-based detection
- Handles file moves and renames correctly

3 Key Modules and Responsibilities

This section details each major module in the `ctx` codebase, its responsibilities, and key implementation details.

3.1 CLI Layer

3.1.1 `cli.rs` (278 lines)

Defines the command-line interface structure using the `clap` crate:

- `Args` struct — Global options (format, ignore patterns, verbosity)
- `Command` enum — Top-level subcommands
- `QueryCommand` enum — Query-specific subcommands
- `OutputFormat` enum — Output format options (XML, Markdown, Plain, JSON)

3.1.2 `main.rs` (1,272 lines, 43 symbols)

The application entry point and command dispatcher:

- Entry point with error handling
- Individual `run_*` functions for each command
- Output formatting helpers
- Watch mode implementations

3.2 Context Generation Pipeline

3.2.1 `walker.rs` (230 lines, 14 symbols)

File discovery and filtering using the `ignore` crate:

```
1 let mut builder = WalkBuilder::new(root);
2 builder.git_ignore(true);           // Respect .gitignore
3 builder.add_custom_ignore_filename(".contextignore");
4
5 for entry in builder.build() {
6     // Process file
7 }
```

Listing 4: File walker configuration

Key features:

- Gitignore support via the `ignore` crate
- Custom `.contextignore` file support